

LLM-Guided Rule Discovery for Web Attack Detection

Rahul Tripathi Armaan Oberai Justine Ellery

This attack does not look like a classic injection

```
/tienda1/publico/anadir.jsp?id=2&precio=39a&cantidad=1
```

- Many attacks are structural, not just keyword matches.
- Endpoint-specific rules can catch what generic signatures miss.
- The LLM proposes rules offline; the detector stays deterministic.

This attack does not look like a classic injection

```
/tienda1/publico/anadir.jsp?id=2&precio=39a&cantidad=1
```

No script tag. No obvious SQL keyword. No path traversal.

- Many attacks are structural, not just keyword matches.
- Endpoint-specific rules can catch what generic signatures miss.
- The LLM proposes rules offline; the detector stays deterministic.

This attack does not look like a classic injection

/tienda1/publico/anadir.jsp?id=2&precio=39a&cantidad=1

No script tag. No obvious SQL keyword. No path traversal.



But precio should be numeric.

- Many attacks are structural, not just keyword matches.
- Endpoint-specific rules can catch what generic signatures miss.
- The LLM proposes rules offline; the detector stays deterministic.

This attack does not look like a classic injection

```
/tienda1/publico/anadir.jsp?id=2&precio=39a&cantidad=1
```

No script tag. No obvious SQL keyword. No path traversal.

But precio should be numeric.

Can an LLM help discover this kind of rule automatically?

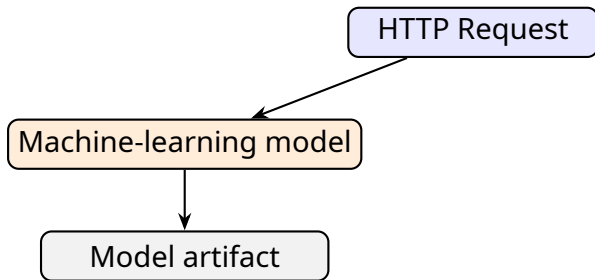
- Many attacks are structural, not just keyword matches.
- Endpoint-specific rules can catch what generic signatures miss.
- The LLM proposes rules offline; the detector stays deterministic.

Accurate detectors are not always deployable rules

HTTP Request

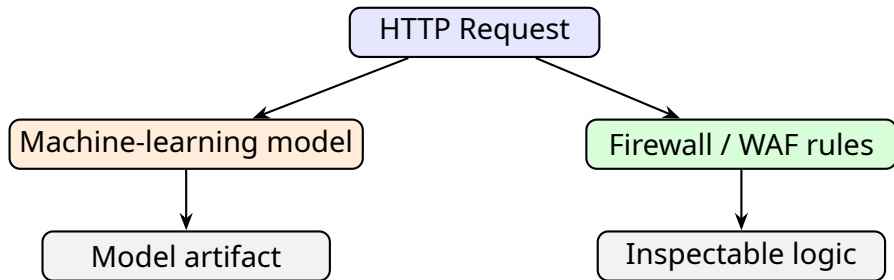
- Web attacks often leave visible evidence in URL, body, and parameters.
- Machine-learning baselines can classify this traffic accurately.
- But many defenses need rules that are fast, auditable, and easy to revise.
- **Question:** can an LLM help discover those rules without running at inference time?

Accurate detectors are not always deployable rules



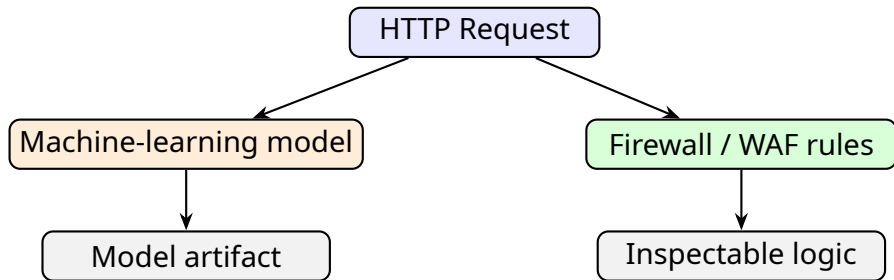
- Web attacks often leave visible evidence in URL, body, and parameters.
- Machine-learning baselines can classify this traffic accurately.
- But many defenses need rules that are fast, auditable, and easy to revise.
- **Question:** can an LLM help discover those rules without running at inference time?

Accurate detectors are not always deployable rules



- Web attacks often leave visible evidence in URL, body, and parameters.
- Machine-learning baselines can classify this traffic accurately.
- But many defenses need rules that are fast, auditable, and easy to revise.
- **Question:** can an LLM help discover those rules without running at inference time?

Accurate detectors are not always deployable rules



- Web attacks often leave visible evidence in URL, body, and parameters.
- Machine-learning baselines can classify this traffic accurately.
- But many defenses need rules that are fast, auditable, and easy to revise.
- **Question:** can an LLM help discover those rules without running at inference time?

Before LLMs, web attack detection split between rules and models

Rule-based systems
WAF / IDS signatures

- Signature systems such as Snort, Suricata, and WAFs make detection logic deployable.
- CSIC 2010 also became a benchmark for TF-IDF, tree models, statistical models, and deep learning.
- The tradeoff is familiar: rules are operationally simple, while learned models often hide the reason for a decision.

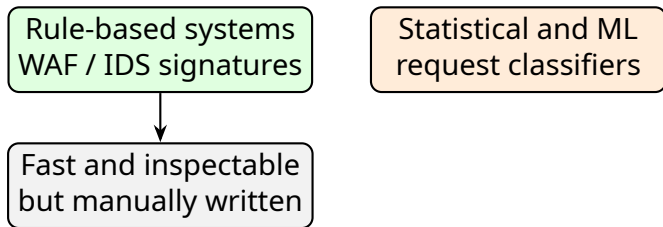
Before LLMs, web attack detection split between rules and models

Rule-based systems
WAF / IDS signatures

Statistical and ML
request classifiers

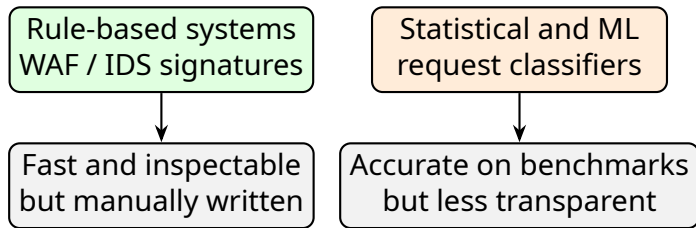
- Signature systems such as Snort, Suricata, and WAFs make detection logic deployable.
- CSIC 2010 also became a benchmark for TF-IDF, tree models, statistical models, and deep learning.
- The tradeoff is familiar: rules are operationally simple, while learned models often hide the reason for a decision.

Before LLMs, web attack detection split between rules and models



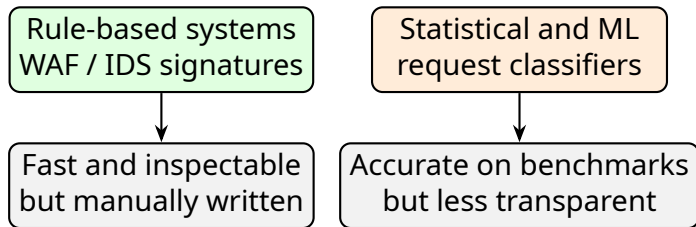
- Signature systems such as Snort, Suricata, and WAFs make detection logic deployable.
- CSIC 2010 also became a benchmark for TF-IDF, tree models, statistical models, and deep learning.
- The tradeoff is familiar: rules are operationally simple, while learned models often hide the reason for a decision.

Before LLMs, web attack detection split between rules and models



- Signature systems such as Snort, Suricata, and WAFs make detection logic deployable.
- CSIC 2010 also became a benchmark for TF-IDF, tree models, statistical models, and deep learning.
- The tradeoff is familiar: rules are operationally simple, while learned models often hide the reason for a decision.

Before LLMs, web attack detection split between rules and models



- Signature systems such as Snort, Suricata, and WAFs make detection logic deployable.
- CSIC 2010 also became a benchmark for TF-IDF, tree models, statistical models, and deep learning.
- The tradeoff is familiar: rules are operationally simple, while learned models often hide the reason for a decision.

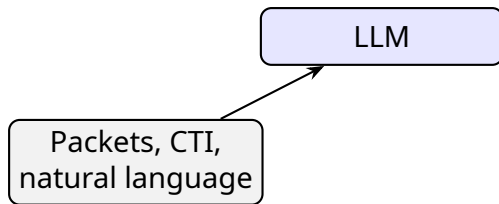
LLMs changed rule generation, but often target heavy rule languages



LLM

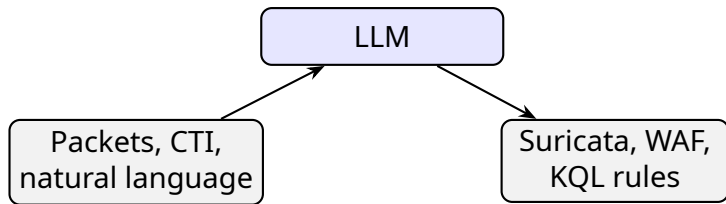
- Recent systems use LLMs to generate IDS, WAF, or EDR detection rules.
- These systems show that LLMs can help translate security context into detection logic.
- They also inherit complex rule grammars, validation needs, latency concerns, and growing rule sets.

LLMs changed rule generation, but often target heavy rule languages



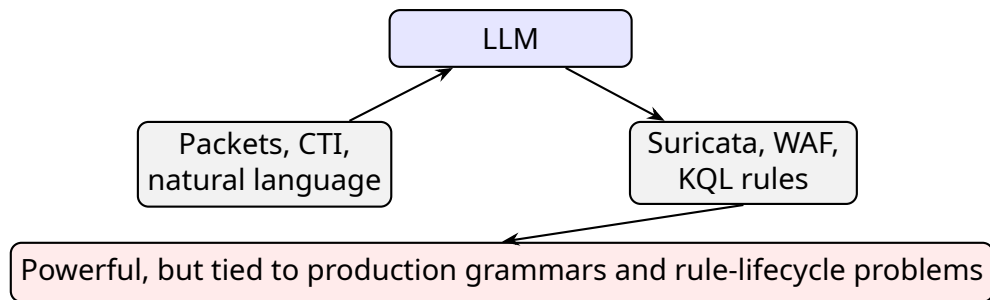
- Recent systems use LLMs to generate IDS, WAF, or EDR detection rules.
- These systems show that LLMs can help translate security context into detection logic.
- They also inherit complex rule grammars, validation needs, latency concerns, and growing rule sets.

LLMs changed rule generation, but often target heavy rule languages



- Recent systems use LLMs to generate IDS, WAF, or EDR detection rules.
- These systems show that LLMs can help translate security context into detection logic.
- They also inherit complex rule grammars, validation needs, latency concerns, and growing rule sets.

LLMs changed rule generation, but often target heavy rule languages



- Recent systems use LLMs to generate IDS, WAF, or EDR detection rules.
- These systems show that LLMs can help translate security context into detection logic.
- They also inherit complex rule grammars, validation needs, latency concerns, and growing rule sets.

Our project studies a smaller, controlled form of rule discovery

LLM proposes simple regexes and endpoint heuristics

- We do not use the LLM as the runtime detector.
- We do not require a production IDS grammar to study the core idea.
- The question is narrower: can auditable rules recover much of the performance of learned models on CSIC 2010?

Our project studies a smaller, controlled form of rule discovery

LLM proposes simple regexes and endpoint heuristics

Harness accepts rules only when dev metrics improve

- We do not use the LLM as the runtime detector.
- We do not require a production IDS grammar to study the core idea.
- The question is narrower: can auditable rules recover much of the performance of learned models on CSIC 2010?

Our project studies a smaller, controlled form of rule discovery

LLM proposes simple regexes and endpoint heuristics

Harness accepts rules only when dev metrics improve

Frozen detector is deterministic Python logic

- We do not use the LLM as the runtime detector.
- We do not require a production IDS grammar to study the core idea.
- The question is narrower: can auditable rules recover much of the performance of learned models on CSIC 2010?

Our project studies a smaller, controlled form of rule discovery

LLM proposes simple regexes and endpoint heuristics

Harness accepts rules only when dev metrics improve

Frozen detector is deterministic Python logic

Final comparison uses the same held-out 80/20 test split as the ML baseline

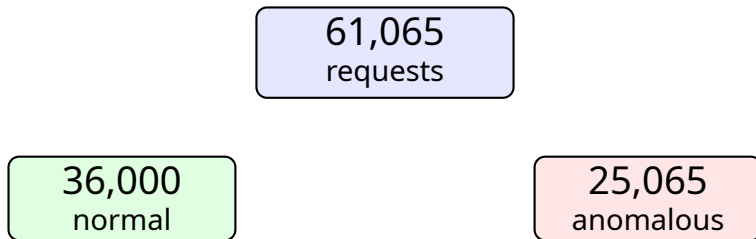
- We do not use the LLM as the runtime detector.
- We do not require a production IDS grammar to study the core idea.
- The question is narrower: can auditable rules recover much of the performance of learned models on CSIC 2010?

CSIC 2010 web-application requests

61,065
requests

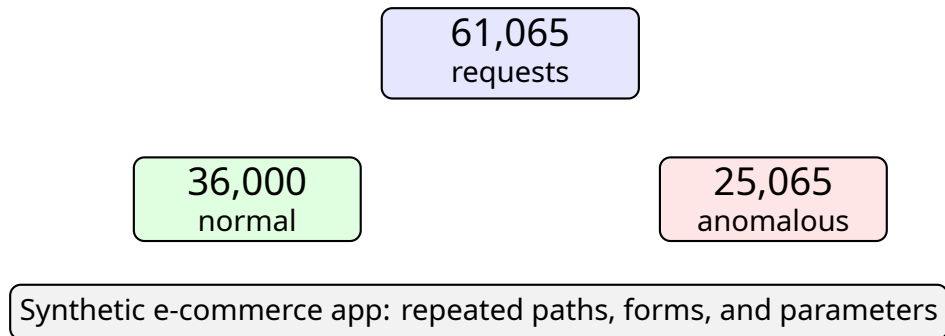
- Attacks include SQL injection, XSS, CRLF injection, traversal, and malformed requests.
- We use a stratified 80/20 split with seed 42.

CSIC 2010 web-application requests



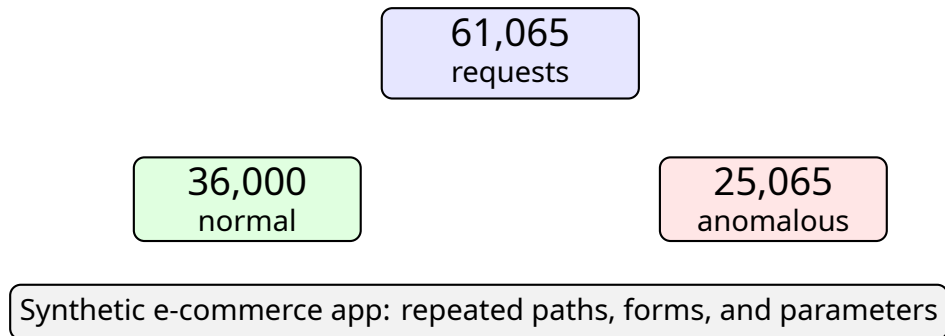
- Attacks include SQL injection, XSS, CRLF injection, traversal, and malformed requests.
- We use a stratified 80/20 split with seed 42.

CSIC 2010 web-application requests



- Attacks include SQL injection, XSS, CRLF injection, traversal, and malformed requests.
- We use a stratified 80/20 split with seed 42.

CSIC 2010 web-application requests



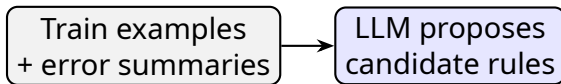
- Attacks include SQL injection, XSS, CRLF injection, traversal, and malformed requests.
- We use a stratified 80/20 split with seed 42.

Use the LLM offline, not in the detector

Train examples
+ error summaries

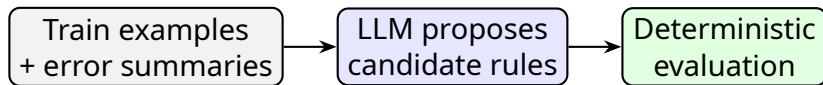
- The LLM proposes regexes and endpoint heuristics.
- Python executes the candidate rules and measures precision, recall, and F1.
- The final detector contains no LLM calls.

Use the LLM offline, not in the detector



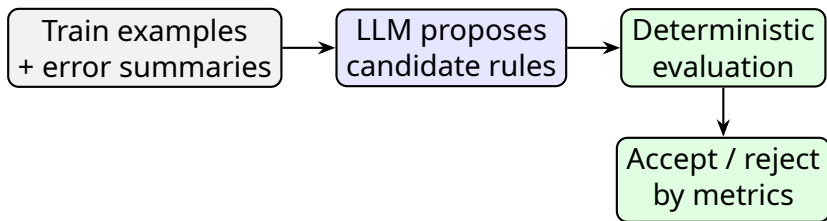
- The LLM proposes regexes and endpoint heuristics.
- Python executes the candidate rules and measures precision, recall, and F1.
- The final detector contains no LLM calls.

Use the LLM offline, not in the detector



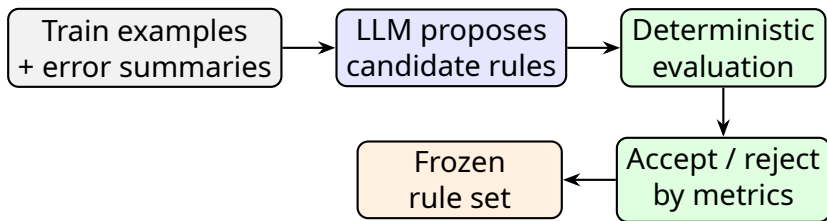
- The LLM proposes regexes and endpoint heuristics.
- Python executes the candidate rules and measures precision, recall, and F1.
- The final detector contains no LLM calls.

Use the LLM offline, not in the detector



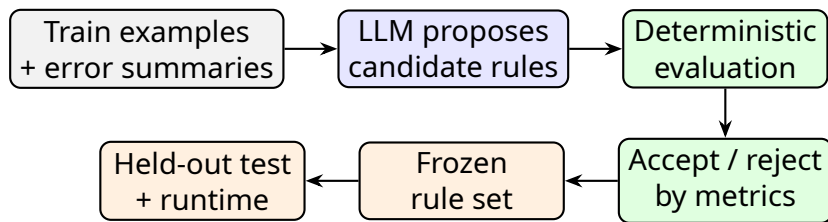
- The LLM proposes regexes and endpoint heuristics.
- Python executes the candidate rules and measures precision, recall, and F1.
- The final detector contains no LLM calls.

Use the LLM offline, not in the detector



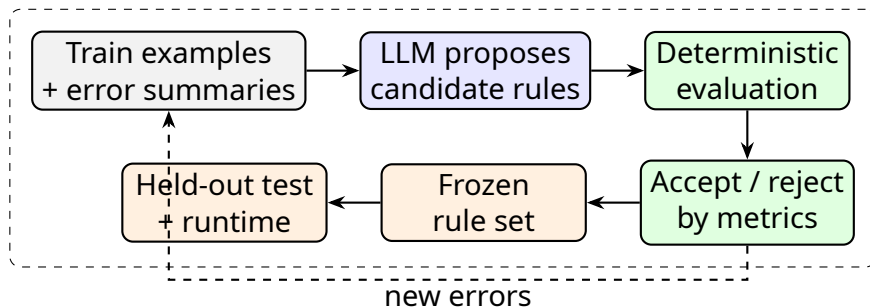
- The LLM proposes regexes and endpoint heuristics.
- Python executes the candidate rules and measures precision, recall, and F1.
- The final detector contains no LLM calls.

Use the LLM offline, not in the detector



- The LLM proposes regexes and endpoint heuristics.
- Python executes the candidate rules and measures precision, recall, and F1.
- The final detector contains no LLM calls.

Use the LLM offline, not in the detector



- The LLM proposes regexes and endpoint heuristics.
- Python executes the candidate rules and measures precision, recall, and F1.
- The final detector contains no LLM calls.

Rule examples: combine attack syntax with endpoint structure

Generic regex rules

- SQL comment markers and keyword clusters
- Script tags and encoded script fragments
- Path traversal strings
- Null bytes and suspicious file probes

Rule examples: combine attack syntax with endpoint structure

Generic regex rules

- SQL comment markers and keyword clusters
- Script tags and encoded script fragments
- Path traversal strings
- Null bytes and suspicious file probes

Endpoint heuristics

- Numeric fields must stay numeric
- Known forms should not gain new fields
- Stable button values should not mutate
- Contact and identity fields have constrained formats

Rule examples: combine attack syntax with endpoint structure

Generic regex rules

- SQL comment markers and keyword clusters
- Script tags and encoded script fragments
- Path traversal strings
- Null bytes and suspicious file probes

Endpoint heuristics

- Numeric fields must stay numeric
- Known forms should not gain new fields
- Stable button values should not mutate
- Contact and identity fields have constrained formats

The biggest gains came after the harness learned application invariants, not just attack strings.

Example: A numeric-field invariant catches subtle mutations

Missed attack example

`/tienda1/publico/anadir.jsp?id=2&precio=39a&cantidad=1`

- The request does not need a classic SQL or XSS token.
- It is anomalous because it violates the endpoint's expected parameter grammar.

Example: A numeric-field invariant catches subtle mutations

Missed attack example

`/tienda1/publico/anadir.jsp?id=2&precio=39a&cantidad=1`



LLM hypothesis

Some endpoint fields should be numeric after URL decoding.

- The request does not need a classic SQL or XSS token.
- It is anomalous because it violates the endpoint's expected parameter grammar.

Example: A numeric-field invariant catches subtle mutations

Missed attack example

`/tienda1/publico/anadir.jsp?id=2&precio=39a&cantidad=1`



LLM hypothesis

Some endpoint fields should be numeric after URL decoding.

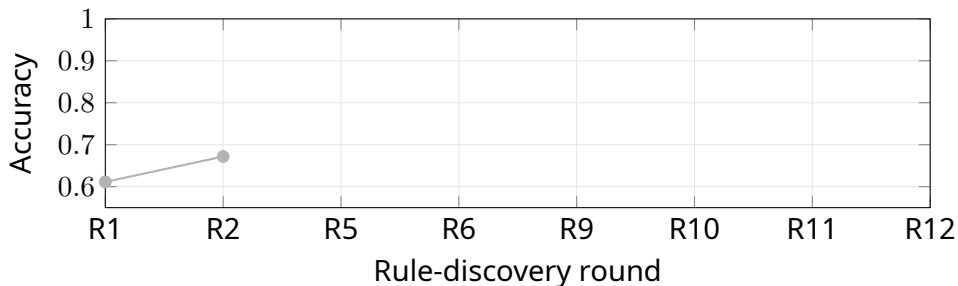


Accepted rule

If `id`, `precio`, or `cantidad` contains a non-numeric value, flag the request.

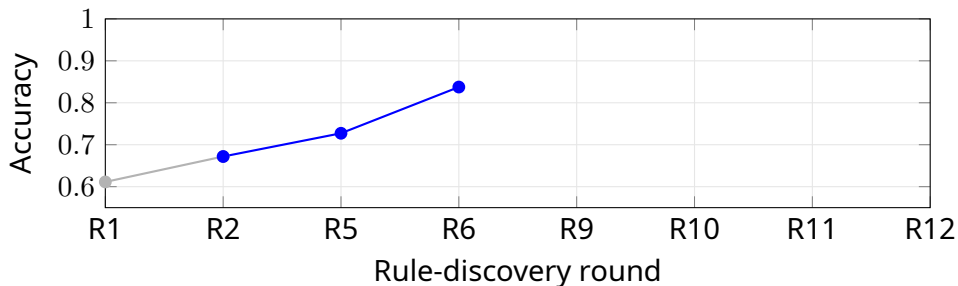
- The request does not need a classic SQL or XSS token.
- It is anomalous because it violates the endpoint's expected parameter grammar.

Accuracy improves across rule-discovery rounds



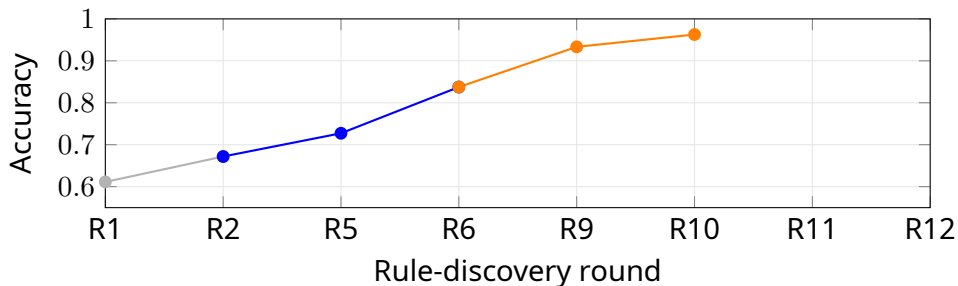
- Seed signatures found obvious attacks but had low recall.
- Endpoint schemas produced the first large jump.
- Encoded payloads and value checks closed much of the gap.
- Final identity/contact heuristics reached 99.21% test accuracy.

Accuracy improves across rule-discovery rounds



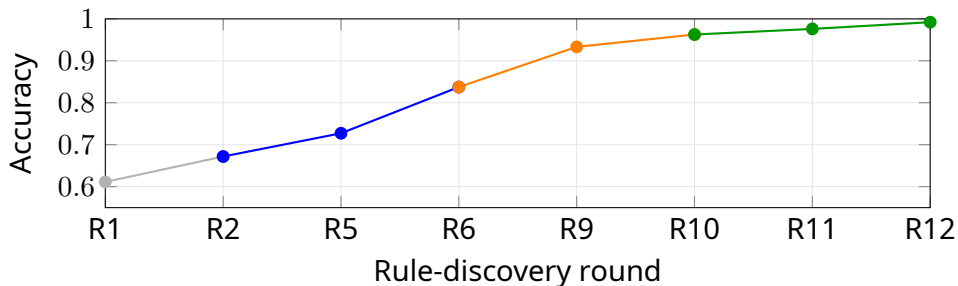
- Seed signatures found obvious attacks but had low recall.
- Endpoint schemas produced the first large jump.
- Encoded payloads and value checks closed much of the gap.
- Final identity/contact heuristics reached 99.21% test accuracy.

Accuracy improves across rule-discovery rounds



- Seed signatures found obvious attacks but had low recall.
- Endpoint schemas produced the first large jump.
- Encoded payloads and value checks closed much of the gap.
- Final identity/contact heuristics reached 99.21% test accuracy.

Accuracy improves across rule-discovery rounds



- Seed signatures found obvious attacks but had low recall.
- Endpoint schemas produced the first large jump.
- Encoded payloads and value checks closed much of the gap.
- Final identity/contact heuristics reached 99.21% test accuracy.

The frozen rule set reaches 99.21% held-out accuracy

99.21%
accuracy

- Evaluation uses the held-out 20% test split.
- High precision means the final rule set rarely flags benign requests.
- The remaining errors are mostly missed attacks, not false alarms.

The frozen rule set reaches 99.21% held-out accuracy

99.21%
accuracy

99.92%
precision

- Evaluation uses the held-out 20% test split.
- High precision means the final rule set rarely flags benign requests.
- The remaining errors are mostly missed attacks, not false alarms.

The frozen rule set reaches 99.21% held-out accuracy

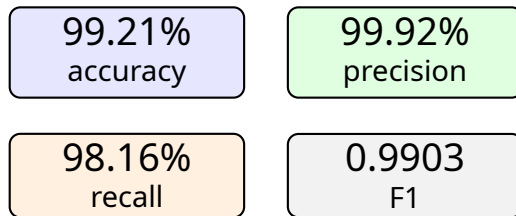
99.21%
accuracy

99.92%
precision

98.16%
recall

- Evaluation uses the held-out 20% test split.
- High precision means the final rule set rarely flags benign requests.
- The remaining errors are mostly missed attacks, not false alarms.

The frozen rule set reaches 99.21% held-out accuracy



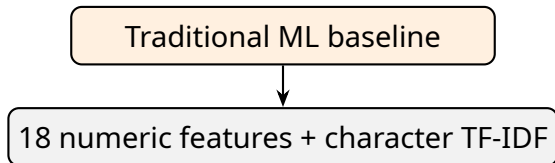
- Evaluation uses the held-out 20% test split.
- High precision means the final rule set rarely flags benign requests.
- The remaining errors are mostly missed attacks, not false alarms.

The baseline is a reproduced random-forest classifier

Traditional ML baseline

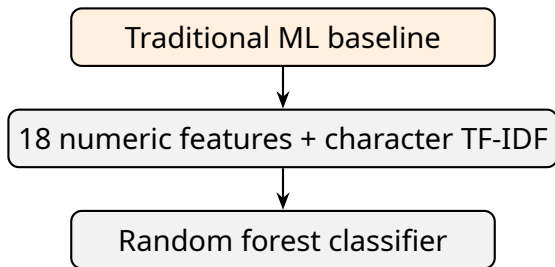
- We reproduced the public CSIC 2010 machine-learning baseline locally.
- The baseline featurizes URL, body, method, and request-length properties.
- The rule harness and ML baseline are reported on the same held-out test split.

The baseline is a reproduced random-forest classifier



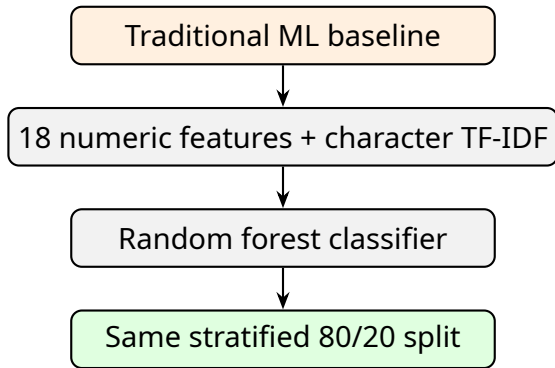
- We reproduced the public CSIC 2010 machine-learning baseline locally.
- The baseline featurizes URL, body, method, and request-length properties.
- The rule harness and ML baseline are reported on the same held-out test split.

The baseline is a reproduced random-forest classifier



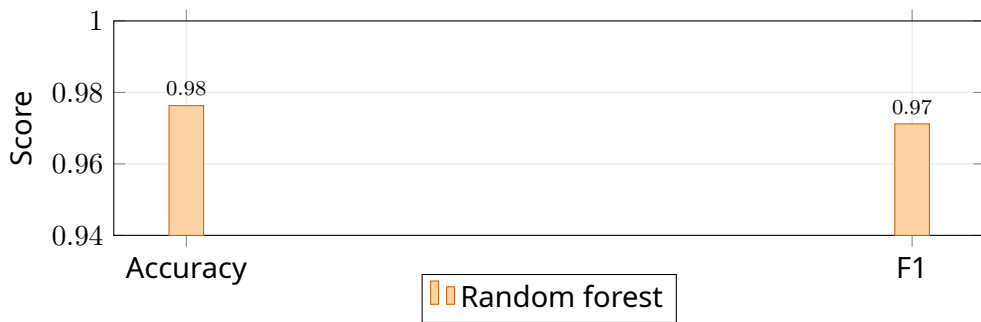
- We reproduced the public CSIC 2010 machine-learning baseline locally.
- The baseline featurizes URL, body, method, and request-length properties.
- The rule harness and ML baseline are reported on the same held-out test split.

The baseline is a reproduced random-forest classifier



- We reproduced the public CSIC 2010 machine-learning baseline locally.
- The baseline featurizes URL, body, method, and request-length properties.
- The rule harness and ML baseline are reported on the same held-out test split.

Traditional ML vs. LLM-guided rules: held-out accuracy and F1



- The reproduced random forest reaches 97.63% accuracy and 0.9712 F1.
- The frozen rule set reaches 99.21% accuracy and 0.9903 F1.
- The takeaway is bounded: on this structured benchmark, compact rules match or exceed a strong text-based classifier.

Traditional ML vs. LLM-guided rules: held-out accuracy and F1



- The reproduced random forest reaches 97.63% accuracy and 0.9712 F1.
- The frozen rule set reaches 99.21% accuracy and 0.9903 F1.
- The takeaway is bounded: on this structured benchmark, compact rules match or exceed a strong text-based classifier.

Traditional ML vs. LLM-guided rules: held-out accuracy and F1



- The reproduced random forest reaches 97.63% accuracy and 0.9712 F1.
- The frozen rule set reaches 99.21% accuracy and 0.9903 F1.
- The takeaway is bounded: on this structured benchmark, compact rules match or exceed a strong text-based classifier.

4 false positives on the held-out test split

- Most remaining errors are false negatives rather than false positives.
- This matters for rule deployment, where noisy rules are hard to keep enabled.

4 false positives on the held-out test split

	Predicted normal	Predicted attack
Actual normal	7,196	4
Actual attack	92	4,921

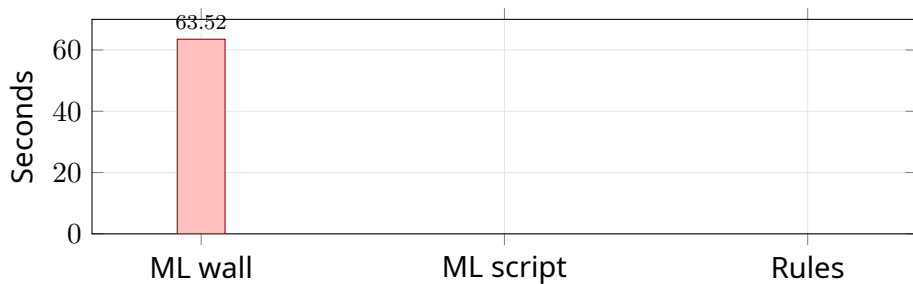
- Most remaining errors are false negatives rather than false positives.
- This matters for rule deployment, where noisy rules are hard to keep enabled.

4 false positives on the held-out test split

	Predicted normal	Predicted attack
Actual normal	7,196	4
Actual attack	92	4,921

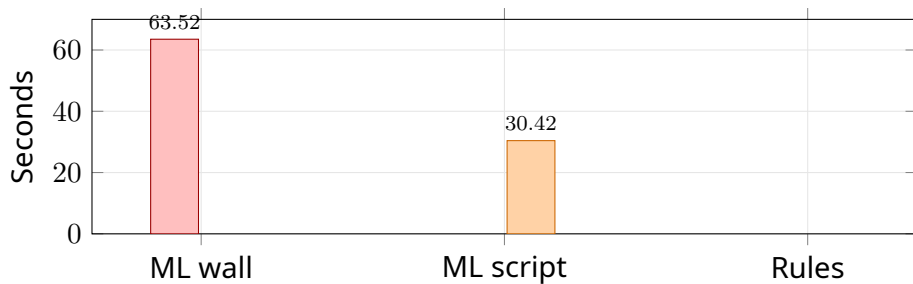
- Most remaining errors are false negatives rather than false positives.
- This matters for rule deployment, where noisy rules are hard to keep enabled.

Runtime comparison: rules remove the heavy pipeline



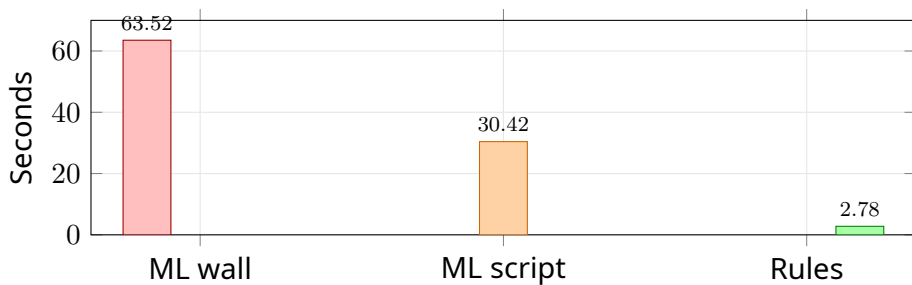
- The wall-clock ML run takes 63.52 seconds for the local pipeline.
- The reproduced ML script timer takes 30.42 seconds for feature extraction, training, and evaluation.
- The frozen rule harness evaluates all 61,065 requests in 2.78 seconds.

Runtime comparison: rules remove the heavy pipeline



- The wall-clock ML run takes 63.52 seconds for the local pipeline.
- The reproduced ML script timer takes 30.42 seconds for feature extraction, training, and evaluation.
- The frozen rule harness evaluates all 61,065 requests in 2.78 seconds.

Runtime comparison: rules remove the heavy pipeline



- The wall-clock ML run takes 63.52 seconds for the local pipeline.
- The reproduced ML script timer takes 30.42 seconds for feature extraction, training, and evaluation.
- The frozen rule harness evaluates all 61,065 requests in 2.78 seconds.

Limitations and next steps

- **Discovery provenance:** rules were developed after earlier inspection of the dataset.
- **Dataset scope:** CSIC 2010 is one synthetic application with repeated endpoint structure.
- **Runtime comparison:** the ML baseline timing is a local pipeline cost, not trained-model inference only.
- **Next step:** freeze the train/test split first, then rerun the agent loop using only training examples and dev errors.
- **Longer term:** test whether the same method works across more web-attack datasets.

Limitations and next steps

- **Discovery provenance:** rules were developed after earlier inspection of the dataset.
- **Dataset scope:** CSIC 2010 is one synthetic application with repeated endpoint structure.
- **Runtime comparison:** the ML baseline timing is a local pipeline cost, not trained-model inference only.
- **Next step:** freeze the train/test split first, then rerun the agent loop using only training examples and dev errors.
- **Longer term:** test whether the same method works across more web-attack datasets.

Limitations and next steps

- **Discovery provenance:** rules were developed after earlier inspection of the dataset.
- **Dataset scope:** CSIC 2010 is one synthetic application with repeated endpoint structure.
- **Runtime comparison:** the ML baseline timing is a local pipeline cost, not trained-model inference only.
- **Next step:** freeze the train/test split first, then rerun the agent loop using only training examples and dev errors.
- **Longer term:** test whether the same method works across more web-attack datasets.

Limitations and next steps

- **Discovery provenance:** rules were developed after earlier inspection of the dataset.
- **Dataset scope:** CSIC 2010 is one synthetic application with repeated endpoint structure.
- **Runtime comparison:** the ML baseline timing is a local pipeline cost, not trained-model inference only.
- **Next step:** freeze the train/test split first, then rerun the agent loop using only training examples and dev errors.
- **Longer term:** test whether the same method works across more web-attack datasets.

Limitations and next steps

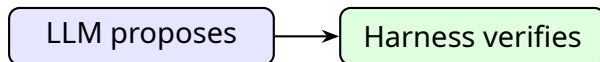
- **Discovery provenance:** rules were developed after earlier inspection of the dataset.
- **Dataset scope:** CSIC 2010 is one synthetic application with repeated endpoint structure.
- **Runtime comparison:** the ML baseline timing is a local pipeline cost, not trained-model inference only.
- **Next step:** freeze the train/test split first, then rerun the agent loop using only training examples and dev errors.
- **Longer term:** test whether the same method works across more web-attack datasets.

LLMs can help write rules without becoming detectors

LLM proposes

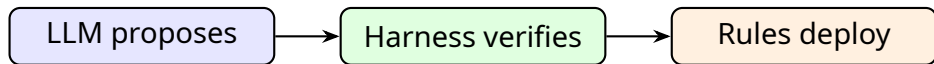
- Use the LLM for offline search, not runtime classification.
- Keep only rules that survive deterministic evaluation.
- Report on held-out data and inspect the remaining errors.
- The result is fast, auditable detection logic for a structured benchmark.

LLMs can help write rules without becoming detectors



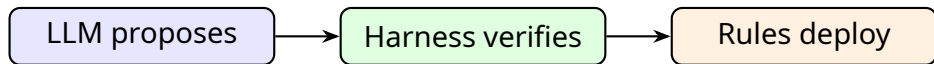
- Use the LLM for offline search, not runtime classification.
- Keep only rules that survive deterministic evaluation.
- Report on held-out data and inspect the remaining errors.
- The result is fast, auditable detection logic for a structured benchmark.

LLMs can help write rules without becoming detectors



- Use the LLM for offline search, not runtime classification.
- Keep only rules that survive deterministic evaluation.
- Report on held-out data and inspect the remaining errors.
- The result is fast, auditable detection logic for a structured benchmark.

LLMs can help write rules without becoming detectors



- Use the LLM for offline search, not runtime classification.
- Keep only rules that survive deterministic evaluation.
- Report on held-out data and inspect the remaining errors.
- The result is fast, auditable detection logic for a structured benchmark.

- LLMs can help discover web-attack rules without serving as runtime detectors.
- The final detector is a compact deterministic rule set over request text and endpoint fields.
- On the held-out 20% split, the rule set reaches 99.21% accuracy and 0.9903 F1.
- The rule harness evaluates all 61,065 requests in 2.78 seconds.
- Compared with the reproduced ML baseline, the rules are faster, more inspectable, and easier to adapt into firewall-style logic.

Q & A